Lesson 2
Basic Frame Operations

Contents

1. Drawing Shapes and Writing Text on a Frame
2. Using Mouse Events to Draw
3. Image Blurring

Useful Links

- OpenCV Documentation: https://docs.opencv.org/4.x/
- OpenCV Python Tutorials: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- Numpy References: https://numpy.org/doc/1.23/reference/index.html#reference

Overview

In most cases, it is often not enough to only simply read an image or frame, before using it later on in a computer vision program for operations such as object detection.

Enhancements to increase usability, such as drawing bounding rectangles around detected objects can be useful to more clearly convey information. Drawing functions also come in useful when trying to debug an OpenCV program, with it being much easier to see what an OpenCV program is doing (i.e. coordinates of a detected object) with visual prompts, rather than trying to read the raw numerical output that other programmers may use (i.e. using *print()*).

Images are often accompanied by a large amount of noise (unwanted data that interferes with the data we are actually interested in) due to the large amount of information contained within even a low-resolution image. A 240p image (240 x 426) contains over 100,000 separate data points for us to analyse.

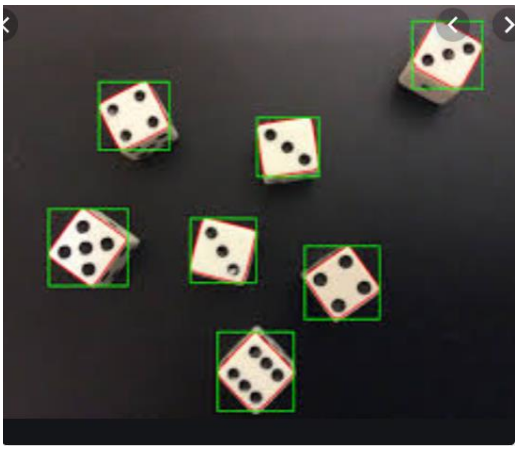


Image blurring which involves the averaging of pixels, lowers the magnitude of noise in an image while maintaining higher magnitude signals which we are generally interested in. Image blurring is often a prerequisite before performing other operations such as edge detection and image thresholding.

Drawing Shapes

OpenCV provides us with a number of drawing functions for a wide variety of different shapes. Most of these functions revolve around the concept of a point containing x and y coordinates.

```
x, y = 350, 200
point1 = (x, y)
```

The number of points required to draw a shape depends on what shape is being drawn. Shapes such as circles and ellipses also take arguments for their radius. All drawing functions take an argument for their colour. Like with a point, colour is usually specified as a tuple. Also, remember that OpenCV handles colours in the Blue, Green and Red colourspace (BGR) and not in RGB.

```
B, G, R = 125, 50, 255
colour = (B, G, R)
```

For all shapes that enclose an area (i.e. not a line), if the thickness argument is set to -1, then the shape will be filled, otherwise, if the thickness is a positive integer it specifies the thickness of the shape's outline.

cv.line(image, pt1, pt2, colour, thickness) -> None

- Draws a line from pt1 to pt2 on the image passed as an argument. The line is in the colour specified by the colour argument.
- Thickness is measured in pixel width.
- Different line styles can be drawn using the lineType argument. Please check the OpenCV documentation for further reference.

cv.rectangle(image, pt1, pt2, colour, thickness) -> None

- Draws a rectangle on the image passed as an argument.
- pt1 and pt2 are opposite vertices of the rectangle being drawn.

cv.circle(image, centre, radius, colour, thickness) -> None

- Draws a circle on the provided image.

cv.polylines(image, points, is_closed, colour, thickness) -> None

- Useful to draw multiple lines at once on a provided image. The points argument is an array of points that are connected together with lines in the order they are in. Pass points in surrounded by square brackets ([points]) otherwise, you will get a numpy dimension error.
- Points must be a 2D numpy array with **np.int32** is it's data type argument.
- is_closed is a boolean value with determines if the polylines are a closed area.
- Useful for drawing shapes such as polygons and trapeziums.

Task 1: Drawing Shapes

1. Create a new python file, importing OpenCV and Numpy.

2. Create a blank image using the Numpy function np.zeros([height, width, 3]). 3 specifies the number of colour channels.

3. Using the functions above, draw one of each shape on the blank canvas with varying colours.

4. Display the image using *cv.imshow().*

Writing Text on a Frame

Often it can be helpful to write text on a frame, whether that is to label bounding rectangles or to include a colour key. OpenCV provides us with a function similar to drawing shapes, which allows us to write text called *cv.putText()*.

cv.putText(image, text, point, font, font_scale, colour, thickness)

- Writes text on the image provided as an argument. The text is passed in as a string.
- The point argument refers to the lower left corner of the string being written.
- See the below link for available fonts. Pass *cv.FONT_OF_YOUR_CHOICE* to access the font. https://docs.opencv.org/4.x/d6/d6e/group__imgproc__draw.html#ga0f9314ea6e35f99bb23f29567fc16e11
- font_scale acts the same as font scaling in text editors such as MS Word.

Task 2: Writing Text

1. Write a string of your choice onto the frame you used in Task 1. Try out different font types, scaling and thicknesses.

2. Have the point set to an x-coordinate that is on the far left of the frame. Observe what happens.

Mouse Events in OpenCV

Mouse events occur whenever the user's mouse does an action, i.e. left/right clicking or moving, etc. Mouse events allow us to add interactivity to our OpenCV programs. To handle mouse events, we must define a function with a specific prototype:

```
def mouse_event_name(event, x, y, flags, *params):
    # Handle mouse event
```

Event is the event type that took place and x and y are the coordinates of the event. Params is an optional argument that allows us to pass our own arguments into the event handling function through *cv.setMouseCallback()*.

cv.setMouseCallback() takes a reference to your mouse event function as an argument, which it calls itself, passing in the arguments seen in the above function prototype. Before we call *cv.setMouseCallback()* we must name the frame we are attaching the mouse event function to, using *cv.namedWindow()*. Please inspect the code below to see this in action.

```
import cv2 as cv
import numpy as np

x0, y0 = None, None         # The initial coordinates of the moues event
drawing = False

def draw_rectangle(event, x, y, flags, *params) -> None:
    """
    Mouse event handling function.
    Draws a rectangle on mouse event.
    """
    global x0, y0, drawing

    image, = params                         # Unpack params tuple
    colour = [0,0,255]                      # Green

    # Start drawing rectangle on down click
    if event == cv.EVENT_LBUTTONDOWN:
        drawing = True
        x0, y0 = x, y

    pt1, pt2 = (x0, y0), (x, y)

    # Expand rectangle size if mouse is moved while downclicked
    if event == cv.EVENT_MOUSEMOVE and drawing:
        cv.rectangle(image, pt1, pt2, colour, -1)
```

```
    # End drawing when downclick released
    elif event == cv.EVENT_LBUTTONUP and drawing:
        drawing = False
        cv.rectangle(image, pt1, pt2, colour, -1)

    return None

def main() -> None:
    blank = np.zeros((720, 720, 3), dtype=np.uint8)       # Create 720x720 blank image
with 3 colour channels

    cv.namedWindow("Image")              # Name current window “Image”
    cv.setMouseCallback("Image", draw_rectangle, param=(blank))

    while True:
        cv.imshow("Image", blank)
        key = cv.waitKey(1)

        if key == ord('d'):
            break

    cv.destroyAllWindows()

if __name__ == "__main__":
    main()
```

Task 3: Drawing a line with mouse events

1. Create a new python file and copy over the above code.

2. Alter the code so that it draws a line instead of a rectangle.

3. **Extension -** Alter the code so that at the push of a keyboard key, the mouse event draws either a rectangle or line.

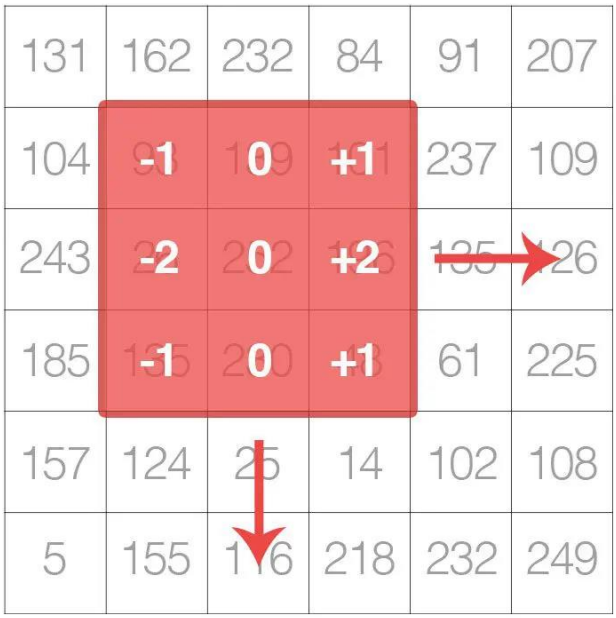
Image Blurring

Although it may sound counterintuitive, image blurring can help us filter out noise in an image, while still being able to detect prominent features. Noise is essentially any data in an image we are not concerned about. Noise can also refer to external factors which can alter how data is presented in an image, such as changes in brightness due to ambient light.

If we are trying to detect the soccer ball and children's playset in the below image, the grass and trees in the background may be considered noise. The light glare coming off the top of the umbrella also acts as noise and can be particularly annoying when trying to threshold for specific colours. The effect of noise on image processing will become more apparent once we start working with edge detection, contours and thresholding in the next lesson.

OpenCV provides us with a number of blurring functions which take advantage of different statistical methods. Each of these functions has its pros and cons and it is up to you to decide which filter is best to use in a given scenario. In essence, blurring involves taking the average of the pixels sitting inside a matrix called a kernel.

The Kernel



A kernel is a matrix used to apply operations to pixels on an image in OpenCV. Only pixels that lay within the boundaries of the kernel are considered for blurring (averaging), with the central pixel being assigned the result of the averaging operation. As a result, increasing the size of the kernel means we take the average over a larger sample area. The kernel also contains weights as its entries, which are used to take weighted averages when required.

Imagine we have a grayscale image with the intensity of pixels defined as a number from 0 to 255 (black to white). Say we want to blur the image to reduce noise with a 3 x 3 sized kernel (seen in red), we will need to follow the below steps:

1. Determine which pixels lie within the kernel.
2. Multiply the pixels within the kernel by the kernel weights that they correspond with.
3. Find the average by summing the multiplied pixel values and dividing by the number of elements inside the kernel (9 in our case).
4. Assign the result to the centre pixel.

If you are still unsure what a kernel is computerphile has made a good video on the topic: https://www.youtube.com/watch?v=C_zFhWdM4ic

Kernels are not just used for blurring and we will see them again shortly when we cover morphological image processing.

cv.blur(image: cv.Mat, kernel_size: tuple) -> cv.Mat

- Applies an standard average blur to an image. `kernel_size` is a tuple in the form width by height.
- All weights in the kernel are set to one.
- Returns the blurred image.

cv.GaussianBlur(image: cv.Mat, kernel_size: tuple, x_s, y_s) -> cv.Mat

- Sets the weights of the kernel to reflect a normal distribution. Subsequently takes a weighted average of the pixels.
- Due to the random nature of most noise, a Gaussian blur is appropriate for most scenarios and is often the default choice for blurring.
- The standard kernel size is (5,5), however, feel free to tinker with the size. <u>The kernel size must have odd dimensions</u>.
- x_s and y_s stand for the standard deviation in the x and y direction respectively. If y_s is not set or is zero, it will be the same value as x_s. If both x_s and y_s are set to zero, then the standard deviation is calculated from the kernel size automatically.
- Returns the blurred image.

cv.medianBlur(image: cv.Mat, kernel_size: int) -> cv.Mat

- Takes the median value contained within the kernel with specified size.
- Best when dealing with “salt and pepper” noise and scratches, which are common in old photographs/films.
- The standard kernel size is 5, however, this can change depending on the level of noise that you wish to filter.

cv.bilateralFilter(image: cv.Mat, d: int, colour_s: int, space_s) -> cv.Mat

- More advanced Gaussian style blur which is great at preserving object edges.
- More powerful than other blurring techniques, however, takes longer to run.
- colour_s is the standard deviation of colour intensities, while space_s is the standard deviation for how far a pixel is from the centre pixel. The greater space_s is, the more of an effect pixels further away have.

cv.filter2D(image: cv.Mat, ddepth: int, kernel: np.ndarray) -> cv.Mat

- Allows us to apply our own kernel to blur an image.
- ddepth is the data type of the returned image object. Set depth to -1 to retain the data type of the input image.

Task 4: Comparing Blurs

1. Create a new python file importing the OpenCV library and matplotlib.pyplot. Download the image of the backyard on page 6 or find an image of your own.

2. Apply a regular blur, gaussian blur, median blur and bilateral blur to the image, assigning them to their own variables. Make sure to use an appropriate kernel size.

3. Create a subplot with dimensions (3 x 2) using matplotlib, recalling the function *plt.imshow()*. Display the original along side the blurred images making sure to use the function *plt.tight_layout()* before using *plt.show()*. Remove the x and y ticks or each subplot calling *plt.xticks([ ])* and *plt.yticks([ ])*.

<u>Example Code for subplots</u>

```
# Plot blurs
    plt.subplot(231), plt.imshow(garden), plt.title("Original")
    plt.xticks([]), plt.yticks([])

    plt.subplot(232), plt.imshow(avg_blur), plt.title("Average")
    plt.xticks([]), plt.yticks([])

    plt.subplot(233), plt.imshow(gaussian), plt.title("Gaussian")
    plt.xticks([]), plt.yticks([])

    plt.subplot(234), plt.imshow(median), plt.title("Median")
    plt.xticks([]), plt.yticks([])

    plt.subplot(235), plt.imshow(bilateral), plt.title("Bilateral")
    plt.xticks([]), plt.yticks([])

    plt.suptitle("Types of Blur")
    plt.tight_layout()
    plt.show()
```

Task 5: Video Blurring

1. Choose one of the blurs from Task 4 and apply it to video frames. Don't use matplotlib for this task as it will be too slow, use *cv2.imshow()* instead.